

A Mini project on
Real-Time Network Intrusion Detection System
submitted in partial fulfillment for award of the degree of Bachelor of Technology in
Information Technology

By

N.Sandhya

23321A1285

B.Sowmya

23321A1298

Ch.Srigowri

23321A12A5

Under the Esteemed Guidance of

Internal Guide

D Navaneetha

Associate Professor, Information Technology



Bhoj Reddy Engineering College for Women

Department of Information Technology

(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade, Approved by AICTE and Affiliated to JNTUH)
Vinaynagar, IS Sadan Crossroads, Saidabad, Hyderabad – 500 059, Telangana. www.brecw.ac.in
Ph: +91 -40-2453 7282; Email: principal@brecw.ac.in

AY 2025–26



Bhoj Reddy Engineering College for Women

Department of Information Technology

(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade, Approved by AICTE and Affiliated to JNTUH)

Recognized by UGC under section 2(f) of the UGC Act, 1956

Vinaynagar, IS Sadan Crossroads, Saidabad, Hyderabad – 500 059, Telangana. www.brecw.ac.in



Ref No: BRECW/IT Dept/Project 2025-26/MP-B-03

Date: 31-03-2026

CERTIFICATE

This is to certify that the Mini Project entitled “**Real-Time Network Intrusion Detection System**” is a bonafide work carried out by

N.Sandhya

23321A1285

B.Sowmya

23321A1298

Ch.Srigowri

23321A12A5

In partial fulfillment for award of the degree of Bachelor of Technology in Department of Information Technology from Bhoj Reddy Engineering College for Women, Hyderabad affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH).

D Navaneetha
Internal Guide

Dr M Sandhya Rani
Head of the Department

External Examiner

ACKNOWLEDGEMENT

It is our pleasure to express our whole hearted thanks to our internal guide **D Navaneetha, Associate Professor, Department of Information Technology** , for her extreme guidance and support in completing this project successfully.

We are thankful to our project coordinator **D Navaneetha, Associate Professor, Department of Information Technology**, for her dynamic valuable guidance and constant management.

We express thanks and gratitude to **Dr M Sandhya Rani, Professor & Head of the Department, Information Technology**, for his encouragement and guidance in carrying out the major project presentation.

We would also like to thank **Dr Shyama Chandra Prasad, Professor & Principal of Bhoj Reddy Engineering College for Women** for encouragement in carrying out our mini project successfully. We are also thankful to the staff members of Information Technology department, my friends and to our parents who helped us in completing this project successfully.

By

N.Sandhya (23321A1285)

B.Sowmya (23321A1298)

Ch.Srigowri (23321A12A5)

Index

Contents	Page No
List of Figures/Screenshots	i
Abstract	ii
1. Introduction	1-3
1.1 Purpose of the Project	2
1.2 Existing System	3
1.3 Proposed System	3
1.4 Advantages of Proposed System	3
2. Related Work	4-5
2.1 Survey	5
3. Requirement Analysis	6-9
3.1 Functional Requirements	6
3.2 Non Functional Requirements	7-8
3.3 Computational Requirements	9
3.3.1 Software Requirements	9
3.3.2 Hardware Requirements	9
4. Design	10-20
4.1 Architecture	11-14
4.1.1 System Architecture	11-12
4.1.2 Technical Architecture	13-14
4.2 UML Diagrams	15-20
4.2.1 Use Case Diagram	15
4.2.2 Class Diagram	16
4.2.3 Sequence Diagram	17
4.2.4 Activity Diagram	18-19
4.3 Modules	20
5. Implementation	21-33
5.1 Pseudo Code	22-33

6. Screenshots	34-39
7. Test Cases	40-41
8. Conclusion	42-43
9. Future Scope	44-45
10. References	46-47

List of Figures

S. No	Figure Name	Fig. No	Page No
1	System Architecture	4.1.1.1	12
2	Technical Architecture	4.1.2.1	14
3	Use case	4.2.1.1	15
4	Class diagram	4.2.2.1	16
5	Sequence diagram	4.2.3.1	17
6	Activity diagram	4.2.4.1	19
7	Screenshots	6.1-6.10	35-39

Abstract

This project presents the design and development of a Real-Time Network Intrusion Detection System (NIDS) using deep learning techniques to enhance cybersecurity in modern network environments. With the rapid expansion of network-based applications and cloud services, traditional intrusion detection systems, which rely on signature-based and rule-based methods, are no longer sufficient to detect sophisticated and evolving cyber threats such as DDoS attacks, SQL injection, port scanning, and malware intrusions.

To overcome these limitations, the proposed system utilizes a Long Short-Term Memory (LSTM) based deep neural network to analyze sequential network traffic patterns. The system continuously monitors network activity, detects anomalies, and classifies traffic into normal or malicious categories with high accuracy. Unlike conventional systems, it is capable of identifying both known and unknown (zero-day) attacks.

The system also provides real-time alerts, visual dashboards for traffic analysis, and supports cloud-based deployment for scalability and efficient performance. Additionally, it maintains a historical database of detected attacks for further analysis and decision-making.

Overall, the proposed solution improves detection accuracy, reduces false positives, and ensures real-time, scalable, and intelligent intrusion detection, making it suitable for modern distributed and cloud-based network environments.

1.Introduction

1.Introduction

In today's digital world, computer networks play a vital role in communication, business operations, and data sharing. With the rapid growth of internet usage, cloud computing, and online services, networks have become increasingly vulnerable to various cyber threats such as Distributed Denial of Service (DDoS) attacks, malware, SQL injection, and unauthorized access.

To protect networks from such threats, Intrusion Detection Systems (IDS) are used to monitor and analyze network traffic for suspicious activities. However, traditional IDS techniques, which rely on signature-based and rule-based methods, are limited in their ability to detect new and evolving attacks. These systems often fail to identify complex attack patterns and generate high false alarms, making them less effective in modern network environments.

To address these challenges, this project introduces a Real-Time Network Intrusion Detection System (NIDS) using deep learning techniques. The proposed system utilizes a Long Short-Term Memory (LSTM) neural network, which is capable of analyzing sequential data and detecting complex patterns in network traffic. This enables the system to accurately identify both known and unknown (zero-day) cyber attacks.

The system continuously monitors network activity, processes large volumes of data in real time, and generates instant alerts when malicious behavior is detected. It also provides visualization tools to analyze attack trends and supports cloud-based deployment for improved scalability and performance. Overall, this project aims to develop an intelligent, efficient, and scalable intrusion detection system that enhances network security and provides reliable protection against modern cyber threats.

1.1 Purpose of the Project:

The purpose of this project is to develop a Real-Time Network Intrusion Detection System (NIDS) that enhances the security of computer networks. With the increasing number of cyber threats such as DDoS attacks, malware, and unauthorized access, traditional intrusion detection systems are not sufficient to detect complex and evolving attacks effectively. This project uses a deep learning approach (LSTM model) to analyze network traffic and identify both known and unknown attacks in real time. The system aims to improve detection accuracy, reduce false alerts, provide instant notifications, and support scalable cloud-based deployment for efficient and reliable network security.

1.2 Existing System:

The existing intrusion detection systems mainly use signature-based and rule-based approaches to detect cyber attacks. These systems compare network traffic with predefined patterns or known attack signatures to identify intrusions. While they are effective in detecting previously known threats, they are not capable of identifying new or unknown (zero-day) attacks. Additionally, these systems suffer from several limitations such as high false positive and false negative rates, lack of real-time efficiency, and poor scalability in handling large and dynamic network traffic. They also require frequent updates and manual effort, making them less effective in modern cloud-based and rapidly evolving network environments.

1.3 Proposed System:

The proposed system is a Real-Time Network Intrusion Detection System (NIDS) that uses a deep learning LSTM model to analyze network traffic and detect cyber attacks. It continuously monitors network data, identifies abnormal behavior, and classifies activities as normal or malicious, including unknown (zero-day) attacks. The system provides real-time alerts and visual analysis of network activity, while also supporting cloud-based deployment for scalability and efficient handling of large volumes of data.

Additionally, the system improves detection accuracy by learning sequential patterns in network traffic, reduces false positives and false negatives, and enables faster response and better decision-making for network administrators. It also maintains attack records for future analysis, making it a reliable and efficient solution for modern network security.

1.4 Advantages of Proposed System:

- Detects both known and unknown (zero-day) attacks
- Provides high accuracy using deep learning (LSTM)
- Supports real-time monitoring and detection
- Reduces false positives and false negatives
- Handles large-scale network traffic efficiently
- Scalable and suitable for cloud-based deployment

2. Related Work

2. Related Work

2.1 Survey:

To assess the real-world need for an effective and intelligent network security solution, a structured survey was conducted targeting students, IT professionals, system administrators, and individuals with basic knowledge of computer networks. These respondents represent potential users and stakeholders of a Network Intrusion Detection System. The survey revealed that nearly 72% of participants were aware of cyber attacks such as malware, phishing, and DDoS, but most lacked access to advanced tools to detect or prevent such threats in real time. A significant number of respondents highlighted that existing security tools are either too complex, expensive, or not easily accessible for small-scale users and organizations.

When asked about the effectiveness of current intrusion detection systems, around 65% of respondents stated that traditional systems fail to detect new or unknown attacks and often generate false alerts. Additionally, about 80% of participants expressed interest in a system that can provide real-time monitoring and accurate attack detection using intelligent techniques like deep learning. Many respondents also emphasized the need for features such as real-time alerts, easy-to-understand dashboards, automated analysis, and scalability for handling large network traffic.

Overall, the survey indicates a strong demand for a smart, reliable, and scalable intrusion detection system that can overcome the limitations of existing methods. It validates the need for developing a deep learning-based solution capable of detecting both known and unknown cyber threats efficiently, ensuring better security for modern network environments.

3. Requirement Analysis

3. Requirement Analysis

3.1.Functional Requirements:

The proposed Network Intrusion Detection System (NIDS) is a comprehensive security solution designed to monitor, analyze, and protect network environments from cyber threats. The system supports multiple functionalities such as real-time traffic monitoring, attack detection, and alert generation. It ensures efficient handling of network data using deep learning techniques and provides a user-friendly interface for administrators to monitor and manage network security effectively.

- Real-Time Network Monitoring
- Data Collection and Preprocessing
- Attack Detection and Classification
- Deep Learning Analysis (LSTM Model)
- Real-Time Alerts and Notifications
- Visualization and Reporting Dashboard

3.1 Non-Functional Requirements:

The proposed Network Intrusion Detection System (NIDS) is designed to be a secure, efficient, and scalable solution that ensures continuous monitoring and protection of network environments. The system must provide a simple and user-friendly interface for administrators to easily monitor network activity, view alerts, and analyze security reports without complexity. It should operate smoothly in real-time, handling large volumes of network traffic while maintaining accuracy and performance. Overall, the system should deliver reliability, speed, and security while supporting continuous operation in dynamic network conditions.

- Performance
- Security
- Scalability
- Usability
- Reliability
- Availability

3.2.1.Performance: The system must provide high-speed processing of network traffic and ensure real-time detection of intrusions without delays. Data collection, preprocessing, and analysis should be performed efficiently so that alerts are generated instantly. The system should handle large-scale network data smoothly without performance degradation.

3.2.2.Security: The system must ensure strong protection of network data and system resources. Secure authentication and access control should be implemented to prevent unauthorized access. All data transmission should be encrypted, and sensitive information such as logs and alerts must be stored securely.

3.2.3.Scalability: The system should be capable of handling increasing network traffic and growing data volumes. It must support expansion without affecting performance, making it suitable for both small-scale and large-scale network environments, including cloud-based systems.

3.2.4.Usability: The system should have an easy-to-use interface that allows administrators to monitor network activity, view alerts, and analyze reports effortlessly. The design should be simple, clear, and accessible even for users with basic technical knowledge.

3.2.5.Reliability: The system must operate consistently without failures during continuous monitoring. It should accurately detect intrusions and avoid crashes or data loss. In case of errors, the system should handle them effectively and provide meaningful feedback.

3.2.6. Availability: The system should be available 24/7 for continuous network monitoring. It must ensure minimal downtime and remain operational even under heavy traffic conditions or minor system failures, ensuring uninterrupted security services.

3.3 Computational Resource

3.3.1 Software Requirements:

- Front-end : React.js , Flutter
- Back-end : Django
- Database : MongoDB
- Development Tool : Visual Studio Code

3.3.2 Hardware Requirements:

- Processor : i5 or above
- RAM : 8 GB or more
- Hard Disk : 20 GB or more

4. Design

4. Design

4.1 Architecture:

4.1.1 System Architecture:

The system architecture of the Real-Time Network Intrusion Detection System Using Deep Neural Networks and Cloud Analytics is designed as a multi-layered structure that processes network data from input to visualization and alert generation. The system begins with the data input layer, where network traffic is collected either through live packet capture or by uploading CSV files in CICIDS2017 format. This is clearly reflected in the interface, where users can start live capture or upload dataset files for analysis. The system supports batch processing of data (for example, up to 1000 rows per batch), ensuring efficient handling of large-scale network traffic.

Once the data is received, it moves to the data processing layer, where it is cleaned, structured, and converted into network flow features suitable for analysis. This preprocessing step ensures that the raw traffic data is transformed into a format that can be effectively used by the deep learning model. The system uses the CICIDS2017 dataset structure, which includes multiple features representing network behavior, enabling accurate detection of both normal and malicious traffic.

The core of the system lies in the detection engine, which uses a Bidirectional Long Short-Term Memory (BiLSTM) neural network. This model is specifically designed for time-series analysis and can identify patterns in sequential network traffic data. Based on the processed input, the model classifies each record as either benign or a specific type of attack such as DoS. It also generates additional outputs including confidence scores and severity levels, which help in assessing the seriousness of detected threats.

The processed results are then handled by the backend and API layer, which manages communication between the frontend interface and the detection model. This layer is responsible for handling user actions such as file uploads and initiating detection, as well as sending processed results back to the user interface. It ensures smooth execution of real-time detection and integration of different system components.

All detection results are stored in the database layer, which uses MongoDB Atlas as shown in the output. The database stores important details such as timestamps, predicted labels, and confidence scores. This enables the system to maintain a historical record of detected attacks, which is later used for analysis and displayed in the attack history section of the dashboard.

The visualization and dashboard layer presents the processed information to the user in an interactive and understandable format. The dashboard displays key metrics such as total traffic analyzed, number of attacks detected, attack rate, and distribution of attack types. It also shows detailed detection results in tabular form along with graphs for better interpretation of network behavior.

Finally, the system includes an alert and response layer, which generates real-time alerts whenever a threat is detected. These alerts display critical information such as attack type, confidence level, and severity, allowing users to quickly respond to potential security threats. The entire system is accessed through a user-friendly interface that integrates all functionalities including traffic analysis, attack history, and model metrics. Overall, the architecture follows a complete pipeline from data acquisition to intelligent threat detection and real-time visualization, making it a robust and scalable intrusion detection system.

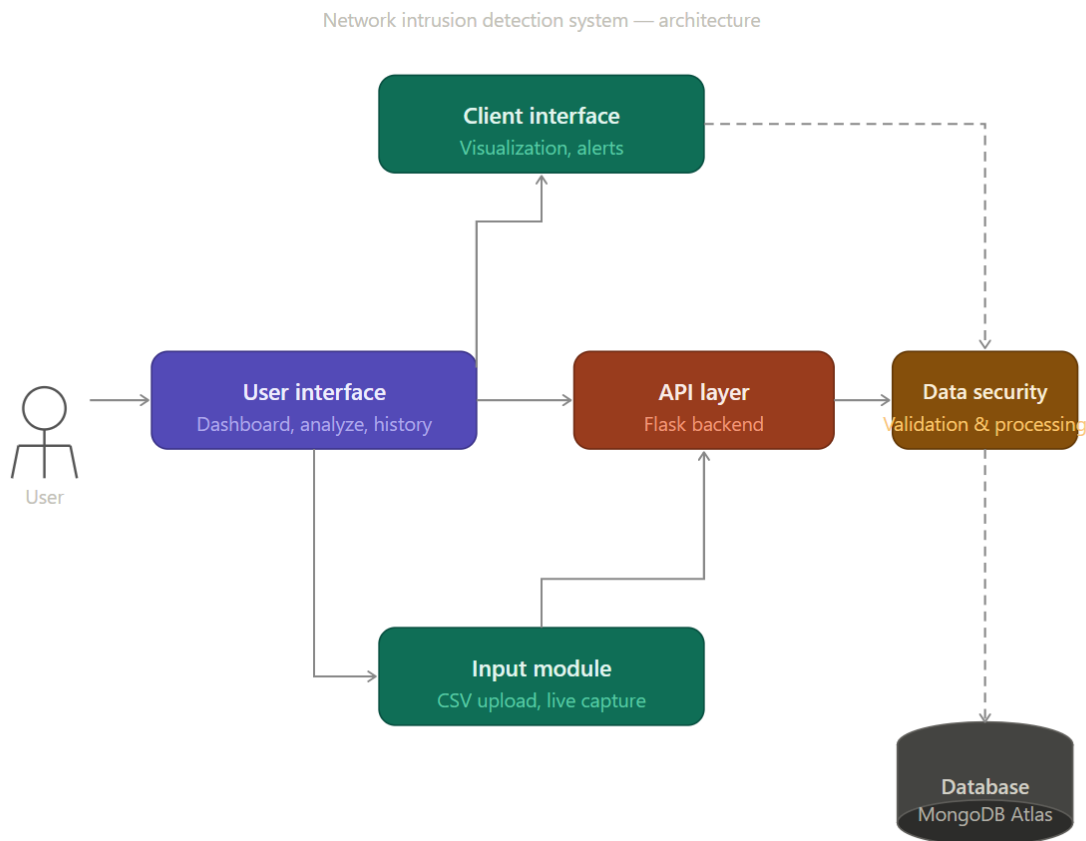


Fig. 4.1.1.1 System Architecture

4.1.2 Technical Architecture:

The technical architecture of the system is designed to enable real-time intrusion detection using an integrated set of web technologies, data processing tools, deep learning models, and cloud storage. The system follows a layered approach to ensure efficient data flow and modular implementation.

The frontend provides a web-based interface with modules such as dashboard, traffic analysis, attack history, and model metrics. It allows users to upload CSV files or initiate live network capture and view results through visualizations and alerts. The frontend communicates with the backend using HTTP and real-time communication.

The backend is implemented using Flask and acts as the central controller. It handles user requests, manages data flow, and coordinates preprocessing and model execution. The data processing layer uses Pandas and NumPy to clean and transform network data from datasets like CICIDS2017, applying feature engineering techniques for better performance.

The core detection is performed using a BiLSTM model built with TensorFlow/Keras, which analyzes sequential network traffic and classifies it into normal or attack types with confidence scores. The system supports both CSV-based batch processing and real-time packet capture using Scapy.

All results are stored in MongoDB Atlas, enabling retrieval of historical data and visualization in the dashboard. Overall, the architecture integrates multiple technologies to provide a scalable and efficient real-time intrusion detection system.

Additionally, the system is designed to be scalable and cloud-deployable, allowing it to handle continuous network data streams efficiently. The modular structure ensures that individual components such as the model, database, or frontend can be upgraded independently without affecting the overall system performance.

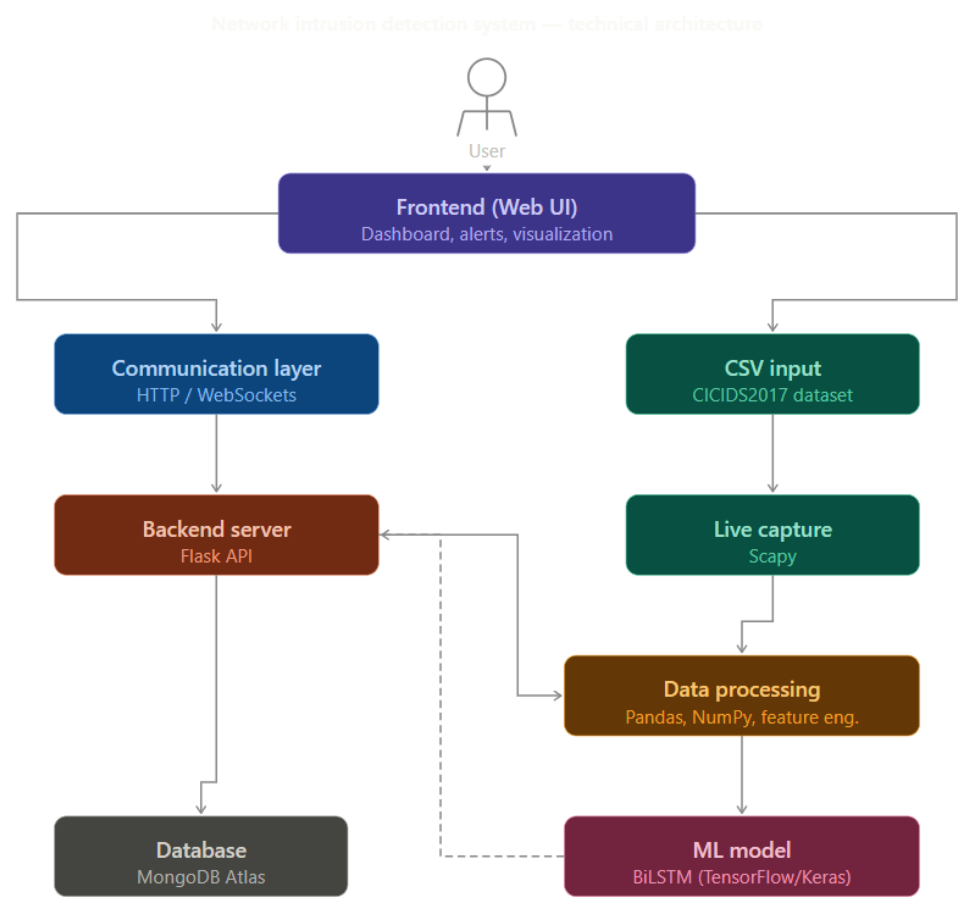


Fig. 4.1.2.1 Technical Architecture

4.2 UML Diagrams

4.2.1 Use Case diagram

A use case diagram is a diagram that shows a set of use cases and actors and their relationships.

4.2.1.1 Common Properties

A use case diagram is just a special kind of diagram and shares the same common properties as do all other diagrams - a name and graphical contents that are a projection into a model. What distinguishes a use case diagram from all other kinds of diagrams is its content.

4.2.1.2 Contents

Use case diagrams commonly contain

- Use cases
- Actors
- Dependency, generalization, and association relationships

4.2.1.3 Common Uses

The use case diagrams are used to model the static use case view of a system. This view primarily supports the behavior of a system - the outwardly visible services that the system provides in the context of its environment.

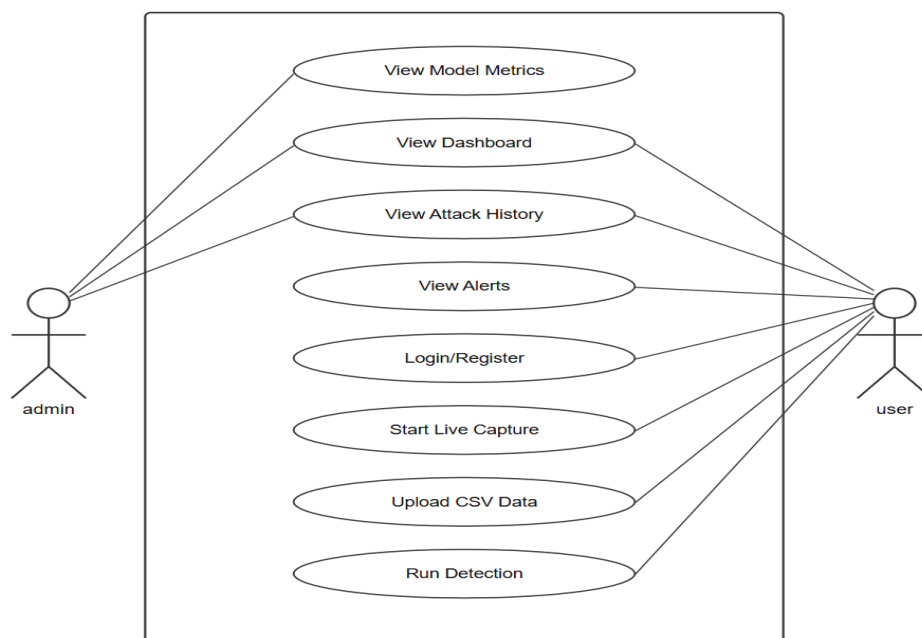


Fig. 4.2.1.1 Use case diagram

4.2.2 Class Diagram:

A class diagram is a diagram that shows a set of classes, interfaces, and collaborations and their relationships.

Class diagram commonly contain the following things:

- Classes
- Interfaces
- Collaborations
- Dependency, generalization and association relationships

4.2.2.1 Common Uses:

Class diagrams are used to model the static design view of a system.

While modelling the static design view of a system, class diagrams are used in one of the three ways:

- To model the vocabulary of a system
- To model simple collaborations
- To model a logical database schema

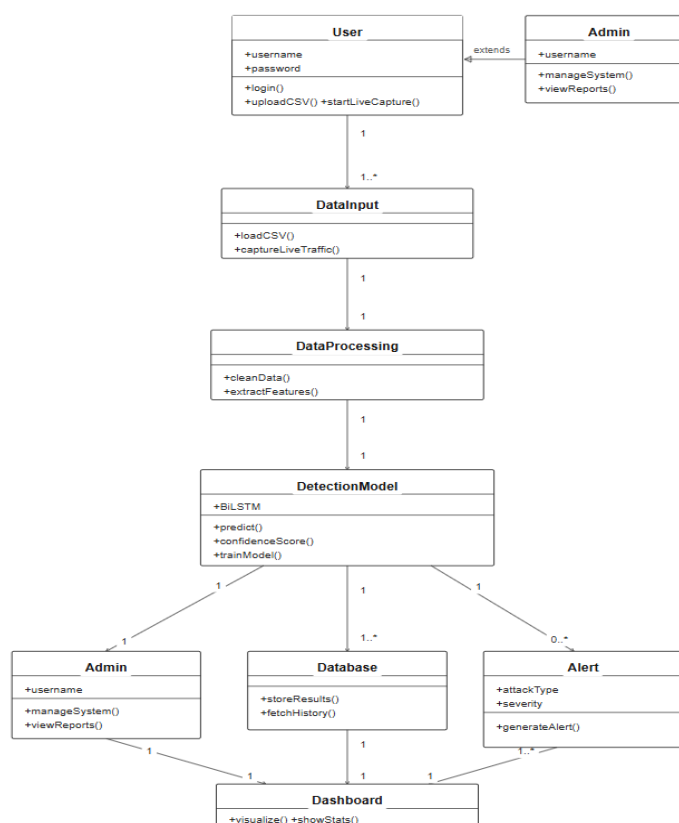


Fig. 4.2.2.1 Class Diagram

4.2.3 Sequence Diagram:

A sequence diagram emphasizes the time ordering of messages. Sequence diagram is formed by first placing the objects that participate in the interaction at the top of your diagram, across the X axis. Typically, you place the object that initiates the interaction at the left, and increasingly more subordinate objects to the right. Next, you place the messages that these objects send and receive along the Y axis, in order of increasing time from top to bottom. This gives the reader a clear visual cue to the flow of control over time. Sequence diagrams have two features that distinguish them from collaboration diagrams. First, there is the object lifeline. An object lifeline is the vertical dashed line that represents the existence of an object over a period of time. Second, there is the focus of control. The focus of control is a tall, thin rectangle that shows the period of time during which an object is performing an action, either directly or through a subordinate procedure.

4.2.4.1 Content:

Sequence diagrams commonly contain

- Objects
- Links
- Messages

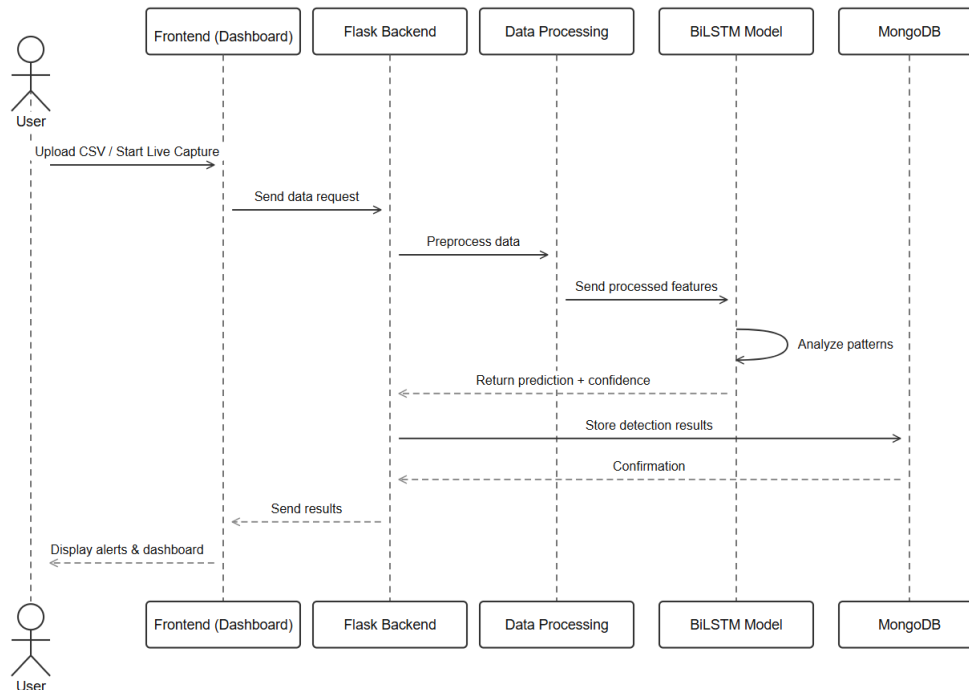


Fig. 4.2.3.1 Sequence Diagram

4.2.4 Activity Diagram:

is made up of executable atomic computations that result in a change in state of the system or the return of a value. Actions encompass calling another operation, sending a signal, creating, or destroying an object, or some pure computation, such as evaluating an expression. Graphically, an activity diagram is a collection of vertices and arcs.

4.2.4.1 Common Properties

An activity diagram is just a special kind of diagram and shares the same common properties as do all other diagrams - a name and graphical contents that are a projection into a model. What distinguishes an interaction diagram from all other kinds of diagrams is its content.

4.2.4.2 Content:

An activity diagram shows the flow from activity to activity. An activity is an ongoing non atomic execution within a state machine.

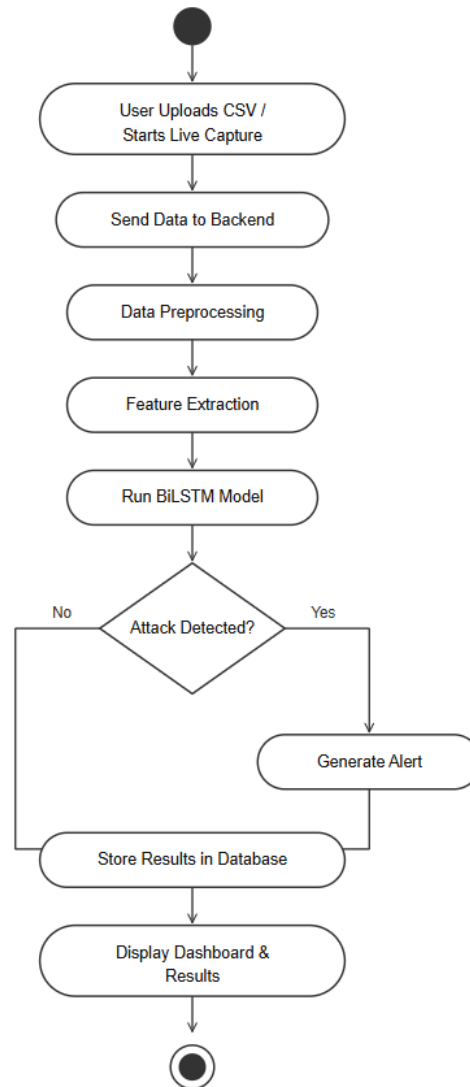
Activities ultimately result in some action, which Activity diagrams commonly contain

- Activity states and action states
- Transitions
- Objects

4.2.4.3 Common Uses:

Activity diagrams are used to model the dynamic aspects of a system. When you model the dynamic aspects of a system, you will typically use activity diagrams in two ways

- To model a workflow
- To model an operation

**Fig. 4.2.4.1 Activity diagram**

4.3 Modules

Module is a part of a program. Programs are composed of one or more independently developed modules. A module description provides detailed information about a module and its supported components. The included description is available directly by making environment check for supported components. The modules are

- **Data Collection & Packet Capture Module :**

Captures real-time network traffic using Scapy and extracts features like IP, protocol, and packet details.

- **Dataset Preprocessing & Feature Engineering Module :**

Prepares datasets (NSL-KDD/CICIDS2017) by cleaning, encoding, normalizing, and splitting data.

- **Deep Learning Model Module (LSTM) :**

Uses LSTM to classify traffic into normal or attack types such as DoS, Probe, R2L, and U2R.

- **Anomaly Detection Module :**

Identifies suspicious traffic based on low confidence or additional methods like Isolation Forest.

- **Real-Time Prediction Engine Module :**

Processes live data, applies the model, and generates real-time attack predictions and alerts.

5. Implementation

5.1 Pseudo code

app.py:

"""

Flask IDS Application - app.py

"""

```
from flask import Flask, render_template, request, jsonify, send_from_directory
from pymongo import MongoClient
from datetime import datetime
import numpy as np
import pandas as pd
import joblib
import os
from datetime import datetime
from collections import Counter
```

```
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
```

```
app = Flask(__name__)
app.secret_key = "ids_secret_2024"
app.config["MAX_CONTENT_LENGTH"] = 500 * 1024 * 1024
```

```
MODEL_FOLDER = "models/"
```

```
print("[INFO] Loading model and artifacts...")
```

```
try:
```

```
    from tensorflow.keras.models import load_model
    model = load_model(os.path.join(MODEL_FOLDER, "ids_lstm_model.h5"), compile=False)
    le = joblib.load(os.path.join(MODEL_FOLDER, "label_encoder.pkl"))
    scaler = joblib.load(os.path.join(MODEL_FOLDER, "scaler.pkl"))
    features = joblib.load(os.path.join(MODEL_FOLDER, "feature_names.pkl"))
    print(f"[INFO] Model loaded. Classes: {list(le.classes_)}")
    MODEL_LOADED = True
```

```
except Exception as e:
```

```
    print(f"[WARNING] Model not loaded: {e}")
    MODEL_LOADED = False
    le = type('obj', (object,), {'classes_': []})()
```

```
ATTACK_LOG = []
```

```
TOTAL_ANALYZED = 0
```

```
# MongoDB Atlas Connection
```

```
MONGO_URI = "mongodb+srv://ids_user:ids_password123@network-ids-cluster.px8vmbk.mongodb.net/?appName=network-ids-cluster"
```

```
client = MongoClient(MONGO_URI)
```

```
db = client["ids_database"]
```

```
attacks_collection = db["attack_logs"]
```

```
print("[INFO] MongoDB Atlas connected successfully!")
```

```

ATTACK_COLORS = {
    "BENIGN": "#4CAF50", "DDoS": "#F44336", "DoS": "#FF5722",
    "Web Attack": "#FF9800", "SQL Injection": "#9C27B0",
    "Port Scan": "#2196F3", "Botnet": "#795548",
    "FTP Brute Force": "#00BCD4", "SSH Brute Force": "#009688",
    "Heartbleed": "#E91E63", "Infiltration": "#607D8B",
}
SEVERITY_MAP = {
    "BENIGN": "Normal", "DDoS": "Critical", "DoS": "Critical",
    "Web Attack": "High", "SQL Injection": "Critical",
    "Port Scan": "Medium", "Botnet": "High",
    "FTP Brute Force": "Medium", "SSH Brute Force": "Medium",
    "Heartbleed": "Critical", "Infiltration": "High",
}

def predict_dataframe(df):
    available = [f for f in features if f in df.columns]
    if len(available) < 5:
        return None, "Not enough matching features. Upload a CICIDS2017 CSV."
    for feat in features:
        if feat not in df.columns:
            df[feat] = 0.0
    X = df[features].values.astype(np.float32)
    X = np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)
    X_scaled = scaler.transform(X)
    X_lstm = X_scaled.reshape(X_scaled.shape[0], 1, X_scaled.shape[1])
    raw_preds = model.predict(X_lstm, verbose=0)
    class_idx = np.argmax(raw_preds, axis=1)
    confidence = np.max(raw_preds, axis=1)
    labels = le.inverse_transform(class_idx)
    results = []
    for i, (label, conf) in enumerate(zip(labels, confidence)):
        results.append({
            "row": i + 1, "label": label,
            "confidence": round(float(conf) * 100, 2),
            "severity": SEVERITY_MAP.get(label, "Unknown"),
            "color": ATTACK_COLORS.get(label, "#607D8B"),
            "timestamp": datetime.now().strftime("%H:%M:%S"),
        })
    return results, None

@app.route("/")
def index():
    stats = {
        "total_analyzed": TOTAL_ANALYZED,
        "total_attacks": sum(1 for e in ATTACK_LOG if e["label"] != "BENIGN"),
        "model_loaded": MODEL_LOADED,
        "attack_types": len(le.classes_) if MODEL_LOADED else 0,
        "recent_log": ATTACK_LOG[-10:][::-1] if ATTACK_LOG else [],
    }

```

```

    }
    return render_template("index.html", stats=stats)

@app.route("/analyze")
def analyze():
    return render_template("analyze.html", model_loaded=MODEL_LOADED)

@app.route("/predict", methods=["POST"])
def predict():
    global TOTAL_ANALYZED, ATTACK_LOG
    if not MODEL_LOADED:
        return jsonify({"error": "Model not loaded. Run python train_model.py first."}), 400
    if "file" not in request.files:
        return jsonify({"error": "No file uploaded"}), 400
    file = request.files["file"]
    if not file.filename.endswith(".csv"):
        return jsonify({"error": "Please upload a CSV file"}), 400
    try:
        df = pd.read_csv(file, low_memory=False)
        df.columns = df.columns.str.strip()
        df.replace([np.inf, -np.inf], np.nan, inplace=True)
        if len(df) > 1000:
            df = df.sample(1000, random_state=42)
        results, error = predict_dataframe(df)
        if error:
            return jsonify({"error": error}), 400
        TOTAL_ANALYZED += len(results)
        ATTACK_LOG.extend(results)
        if len(ATTACK_LOG) > 500:
            ATTACK_LOG = ATTACK_LOG[-500:]
        for r in results:
            attacks_collection.insert_one({
                "timestamp" : datetime.now(),
                "ip_address" : request.remote_addr,
                "attack_type": r["label"],
                "confidence" : r["confidence"],
                "severity"   : r["severity"],
                "source"     : "csv_upload"
            })

        labels = [r["label"] for r in results]
        dist = dict(Counter(labels))
        n_attacks = sum(1 for l in labels if l != "BENIGN")
        return jsonify({
            "success": True, "total_rows": len(results),
            "attacks_found": n_attacks, "benign_count": len(results) - n_attacks,
            "distribution": dist, "results": results[:200],
        })
    except Exception as e:

```

```

        return jsonify({"error": f"Processing error: {str(e)}"}, 500)
@app.route("/history")
def history():
    logs = list(attacks_collection.find().sort("timestamp", -1).limit(100))
    for log in logs:
        log["_id"] = str(log["_id"])
        log["timestamp"] = log["timestamp"].strftime("%Y-%m-%d %H:%M:%S")
    return render_template("history.html", logs=logs)

@app.route("/metrics")
def metrics():
    chart_exists = os.path.exists(os.path.join(MODEL_FOLDER, "training_history.png"))
    cm_exists = os.path.exists(os.path.join(MODEL_FOLDER, "confusion_matrix.png"))
    classes = list(le.classes_) if MODEL_LOADED else []
    return render_template("metrics.html", model_loaded=MODEL_LOADED,
                           chart_exists=chart_exists, cm_exists=cm_exists, classes=classes)

@app.route("/api/stats")
def api_stats():
    if not ATTACK_LOG:
        return jsonify({"total": 0, "attacks": 0, "benign": 0, "distribution": {}})
    labels = [e["label"] for e in ATTACK_LOG]
    return jsonify({
        "total": TOTAL_ANALYZED,
        "attacks": sum(1 for l in labels if l != "BENIGN"),
        "benign": sum(1 for l in labels if l == "BENIGN"),
        "distribution": dict(Counter(labels)),
        "recent": ATTACK_LOG[-5:][::-1],
    })

@app.route("/model_image/<name>")
def model_image(name):
    allowed = {"training_history.png", "confusion_matrix.png"}
    if name not in allowed:
        return "Not found", 404
    return send_from_directory(MODEL_FOLDER, name)
@app.route("/live_capture", methods=["POST"])
def live_capture():
    global TOTAL_ANALYZED, ATTACK_LOG
    if not MODEL_LOADED:
        return jsonify({"error": "Model not loaded."}), 400
    try:
        from capture import capture_live
        duration = int(request.json.get("duration", 30))
        output_file = capture_live(duration=duration)
        if not output_file:
            return jsonify({"error": "No packets captured. Make sure you have network activity."}), 400
        import pandas as pd
        df = pd.read_csv(output_file)

```

```

df.replace([float('inf'), float('-inf')], 0, inplace=True)
results, error = predict_dataframe(df)
if error:
    return jsonify({"error": error}), 400
TOTAL_ANALYZED += len(results)
ATTACK_LOG.extend(results)
labels = [r["label"] for r in results]
from collections import Counter
dist = dict(Counter(labels))
n_attacks = sum(1 for l in labels if l != "BENIGN")
return jsonify({
    "success": True,
    "total_rows": len(results),
    "attacks_found": n_attacks,
    "benign_count": len(results) - n_attacks,
    "distribution": dist,
    "results": results[:200],
})
except Exception as e:
    return jsonify({"error": f"Capture error: {str(e)}"}), 500
if __name__ == "__main__":
    print("\n" + "="*50)
    print(" IDS FLASK APP STARTING...")
    print(" Open browser at: http://127.0.0.1:5000")
    print("="*50 + "\n")
    app.run(debug=True, host="0.0.0.0", port=5000)

```

capture.py:

```

"""
capture.py - Live Packet Capture using Scapy
Captures real network packets and converts to CICIDS2017 features
"""

from scapy.all import sniff, IP, TCP, UDP
import pandas as pd
import numpy as np
from collections import defaultdict
import time
import os

# Storage for flows
flows = defaultdict(list)
flow_stats = []

def get_flow_key(pkt):
    """Create a unique key for each network flow."""
    if IP in pkt:

```

```

src = pkt[IP].src
dst = pkt[IP].dst
proto = pkt[IP].proto
sport = pkt[TCP].sport if TCP in pkt else (pkt[UDP].sport if UDP in pkt else 0)
dport = pkt[TCP].dport if TCP in pkt else (pkt[UDP].dport if UDP in pkt else 0)
return (src, dst, sport, dport, proto)
return None

```

```
def extract_features(pkt_list):
```

```
    """Extract CICIDS2017-like features from a list of packets in a flow."""
```

```
    if not pkt_list:
```

```
        return None
```

```
    lengths = [len(p) for p in pkt_list]
```

```
    times = [p.time for p in pkt_list]
```

```
    # Inter-arrival times
```

```
    iats = [times[i+1] - times[i] for i in range(len(times)-1)] if len(times) > 1 else [0]
```

```
    # TCP flags
```

```
    syn_count = sum(1 for p in pkt_list if TCP in p and p[TCP].flags & 0x02)
```

```
    rst_count = sum(1 for p in pkt_list if TCP in p and p[TCP].flags & 0x04)
```

```
    psh_count = sum(1 for p in pkt_list if TCP in p and p[TCP].flags & 0x08)
```

```
    duration = times[-1] - times[0] if len(times) > 1 else 0.001
```

```
    total_bytes = sum(lengths)
```

```
    features = {
```

```
        'Destination Port'      : pkt_list[0][TCP].dport if TCP in pkt_list[0] else 0,
```

```
        'Flow Duration'         : duration * 1e6,
```

```
        'Total Fwd Packets'     : len(pkt_list),
```

```
        'Total Backward Packets' : 0,
```

```
        'Total Length of Fwd Packets' : total_bytes,
```

```
        'Total Length of Bwd Packets' : 0,
```

```
        'Fwd Packet Length Max'   : max(lengths),
```

```
        'Fwd Packet Length Mean'  : np.mean(lengths),
```

```
        'Bwd Packet Length Mean'  : 0,
```

```
        'Flow Bytes/s'           : total_bytes / duration if duration > 0 else 0,
```

```
        'Flow Packets/s'         : len(pkt_list) / duration if duration > 0 else 0,
```

```
        'Flow IAT Mean'          : np.mean(iats),
```

```
        'Flow IAT Std'           : np.std(iats),
```

```
        'Fwd IAT Total'          : sum(iats),
```

```
        'Bwd IAT Total'          : 0,
```

```
        'Fwd PSH Flags'          : psh_count,
```

```
        'SYN Flag Count'         : syn_count,
```

```
        'RST Flag Count'         : rst_count,
```

```
        'Avg Fwd Segment Size'   : np.mean(lengths),
```

```
        'Init Win bytes forward' : pkt_list[0][TCP].window if TCP in pkt_list[0] else 0,
```

```
    }
```

return features

```
def capture_live(duration=30, output_file="data/live_capture.csv"):
    """
    Capture live packets for `duration` seconds.
    Saves result as CSV to output_file.
    """
    print(f"\n[INFO] Starting live capture for {duration} seconds...")
    print("[INFO] Capturing packets from your network...")
    print("[INFO] Press Ctrl+C to stop early.\n")

    captured = []

    def process_packet(pkt):
        key = get_flow_key(pkt)
        if key:
            flows[key].append(pkt)

    # Capture packets
    sniff(timeout=duration, prn=process_packet, store=False)

    print(f"\n[INFO] Capture done. Processing {len(flows)} flows...")

    for key, pkts in flows.items():
        if len(pkts) >= 2:
            features = extract_features(pkts)
            if features:
                captured.append(features)

    if not captured:
        print("[WARNING] No flows captured. Make sure you have network activity.")
        return None

    df = pd.DataFrame(captured)
    os.makedirs("data", exist_ok=True)
    df.to_csv(output_file, index=False)
    print(f"[INFO] Saved {len(df)} flows to {output_file}")
    return output_file

if __name__ == "__main__":
    capture_live(duration=30)
```

preprocess.py:

=====

STEP 1 - PREPROCESSING SCRIPT

File: preprocess.py
Dataset: CICIDS2017

What this does:

- Loads all CICIDS2017 CSV files
- Cleans column names (removes spaces)
- Handles infinity and NaN values
- Encodes attack labels
- Normalizes features
- Saves cleaned data and encoders

```
=====
"""

import pandas as pd
import numpy as np
import os
import glob
import joblib
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# -----
# CONFIGURATION
# -----
DATA_FOLDER = "data/"      # Place your CICIDS2017 CSVs here
OUTPUT_FOLDER = "models/"  # Saves encoders, scaler here

os.makedirs(OUTPUT_FOLDER, exist_ok=True)

# -----
# ATTACK LABEL MAPPING
# Consolidates similar attacks to cleaner names
# -----
LABEL_MAP = {
    "BENIGN"           : "BENIGN",
    "DDoS"             : "DDoS",
    "DoS Hulk"         : "DoS",
    "DoS GoldenEye"     : "DoS",
    "DoS slowloris"     : "DoS",
    "DoS Slowhttptest"  : "DoS",
    "Heartbleed"        : "Heartbleed",
    "Web Attack Brute Force" : "Web Attack",
    "Web Attack XSS"     : "Web Attack",
    "Web Attack Sql Injection" : "SQL Injection",
    "Infiltration"       : "Infiltration",
    "Bot"                : "Botnet",
    "PortScan"          : "Port Scan",
    "FTP-Patator"       : "FTP Brute Force",
    "SSH-Patator"       : "SSH Brute Force",
```

```

}

# -----
# TOP 20 FEATURES (most important from CICIDS2017)
# Reduces noise and speeds up training
# -----
TOP_FEATURES = [
    'Destination Port',
    'Flow Duration',
    'Total Fwd Packets',
    'Total Backward Packets',
    'Total Length of Fwd Packets',
    'Total Length of Bwd Packets',
    'Fwd Packet Length Max',
    'Fwd Packet Length Mean',
    'Bwd Packet Length Mean',
    'Flow Bytes/s',
    'Flow Packets/s',
    'Flow IAT Mean',
    'Flow IAT Std',
    'Fwd IAT Total',
    'Bwd IAT Total',
    'Fwd PSH Flags',
    'SYN Flag Count',
    'RST Flag Count',
    'Avg Fwd Segment Size',
    'Init_Win_bytes_forward',
]

def load_cicids2017(data_folder):
    """Load all CSV files from the CICIDS2017 dataset folder."""
    csv_files = glob.glob(os.path.join(data_folder, "*.csv"))

    if not csv_files:
        print(f"[ERROR] No CSV files found in '{data_folder}'")
        print(" Please download CICIDS2017 from: https://www.unb.ca/cic/datasets/ids-2017.html")
        print(" And place the CSV files inside the 'data/' folder.")
        return None

    print(f"[INFO] Found {len(csv_files)} CSV files:")
    dfs = []
    for f in csv_files:
        print(f" Loading: {os.path.basename(f)}")
        df = pd.read_csv(f, low_memory=False)
        dfs.append(df)

    combined = pd.concat(dfs, ignore_index=True)
    print(f"[INFO] Total records loaded: {len(combined):,}")

```

```
return combined
```

```
def clean_dataframe(df):
    """Clean and prepare the dataframe."""
    print("\n[STEP 1] Cleaning column names...")
    # CICIDS2017 has spaces before/after column names
    df.columns = df.columns.str.strip()

    print("[STEP 2] Mapping attack labels...")
    label_col = "Label" if "Label" in df.columns else "Label"
    df[label_col] = df[label_col].str.strip()
    df[label_col] = df[label_col].map(LABEL_MAP).fillna(df[label_col])
    df.rename(columns={label_col: "Label"}, inplace=True)

    print("[STEP 3] Replacing infinity values with NaN...")
    df.replace([np.inf, -np.inf], np.nan, inplace=True)

    print("[STEP 4] Dropping rows with NaN values...")
    before = len(df)
    df.dropna(inplace=True)
    print(f" Removed {before - len(df):,} rows with NaN. Remaining: {len(df):,}")

    return df
```

```
def select_features(df):
    """Select only top features that exist in this dataframe."""
    available = [f for f in TOP_FEATURES if f in df.columns]
    missing = [f for f in TOP_FEATURES if f not in df.columns]

    if missing:
        print(f"[WARNING] These features not found in dataset: {missing}")
        print(" Using only available features.")

    print(f"[INFO] Using {len(available)} features for training.")
    return available
```

```
def preprocess_and_save(data_folder=DATA_FOLDER):
    """Main preprocessing pipeline."""
    print("=" * 60)
    print("  CICIDS2017 PREPROCESSING PIPELINE")
    print("=" * 60)

    # 1. Load data
    df = load_cicids2017(data_folder)
    if df is None:
        return
```

```
# 2. Clean
df = clean_dataframe(df)

# 3. Show class distribution
print("\n[INFO] Attack type distribution:")
dist = df["Label"].value_counts()
for label, count in dist.items():
    pct = count / len(df) * 100
    print(f" {label:<25} : {count:>10}, { (pct:.2f)%}")

# 4. Feature selection
features = select_features(df)
X = df[features].values.astype(np.float32)
y = df["Label"].values

# 5. Encode labels
print("\n[STEP 5] Encoding labels...")
le = LabelEncoder()
y_encoded = le.fit_transform(y)
print(f" Classes: {list(le.classes_)}")
joblib.dump(le, os.path.join(OUTPUT_FOLDER, "label_encoder.pkl"))
print(f" Saved: models/label_encoder.pkl")

# 6. Scale features
print("\n[STEP 6] Scaling features with StandardScaler...")
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
joblib.dump(scaler, os.path.join(OUTPUT_FOLDER, "scaler.pkl"))
joblib.dump(features, os.path.join(OUTPUT_FOLDER, "feature_names.pkl"))
print(f" Saved: models/scaler.pkl")
print(f" Saved: models/feature_names.pkl")

# 7. Train/Test split
print("\n[STEP 7] Splitting into train/test (80/20)...")
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

# 8. Reshape for LSTM [samples, timesteps, features]
X_train_lstm = X_train.reshape(X_train.shape[0], 1, X_train.shape[1])
X_test_lstm = X_test.reshape(X_test.shape[0], 1, X_test.shape[1])

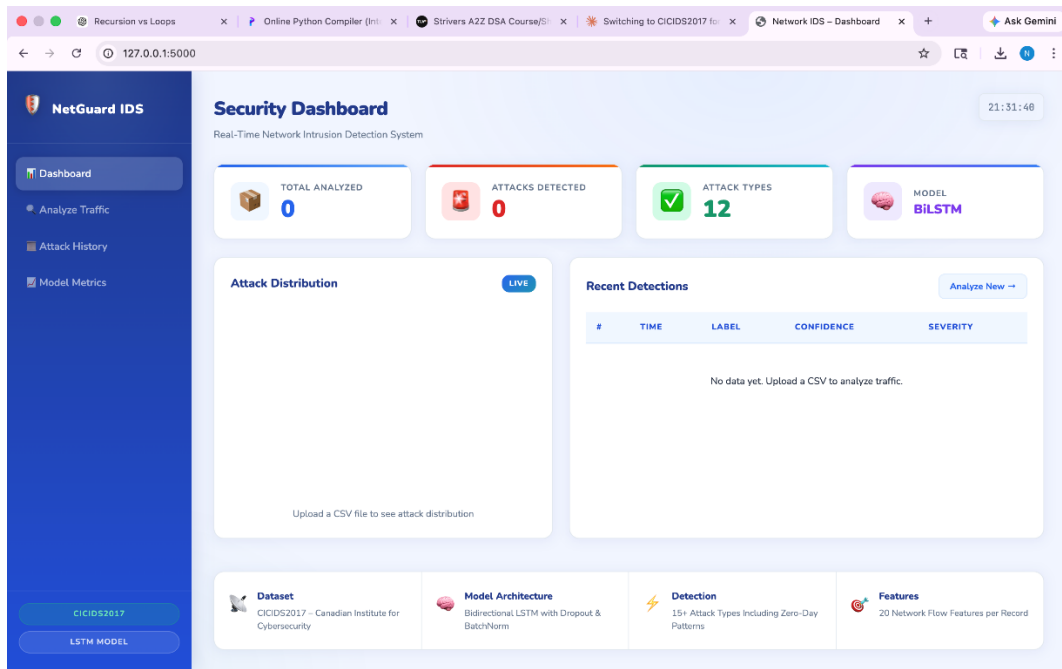
# 9. Save processed arrays
print("\n[STEP 8] Saving processed arrays...")
np.save(os.path.join(OUTPUT_FOLDER, "X_train.npy"), X_train_lstm)
np.save(os.path.join(OUTPUT_FOLDER, "X_test.npy"), X_test_lstm)
np.save(os.path.join(OUTPUT_FOLDER, "y_train.npy"), y_train)
np.save(os.path.join(OUTPUT_FOLDER, "y_test.npy"), y_test)
```

```
print("\n" + "=" * 60)
print(" PREPROCESSING COMPLETE!")
print(f" Train samples : {len(X_train):,}")
print(f" Test samples  : {len(X_test):,}")
print(f" Features used  : {len(features):}")
print(f" Classes       : {len(le.classes_)}")
print("=" * 60)
print("\n Next step: Run  python train_model.py")

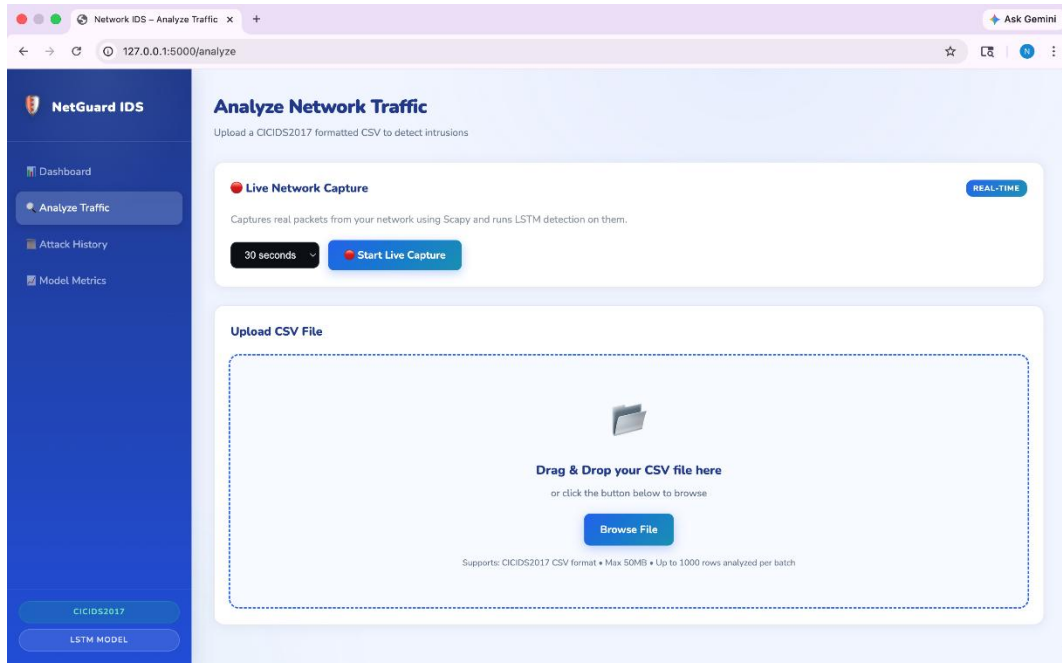
if __name__ == "__main__":
    preprocess_and_save()
```

6. Screenshots

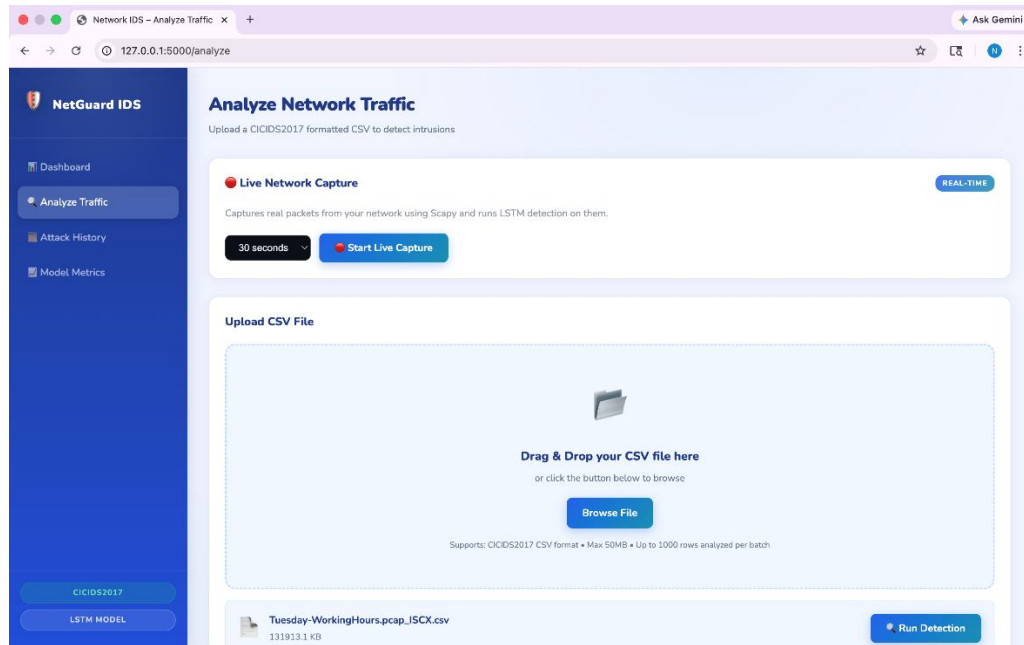
6. Screenshots



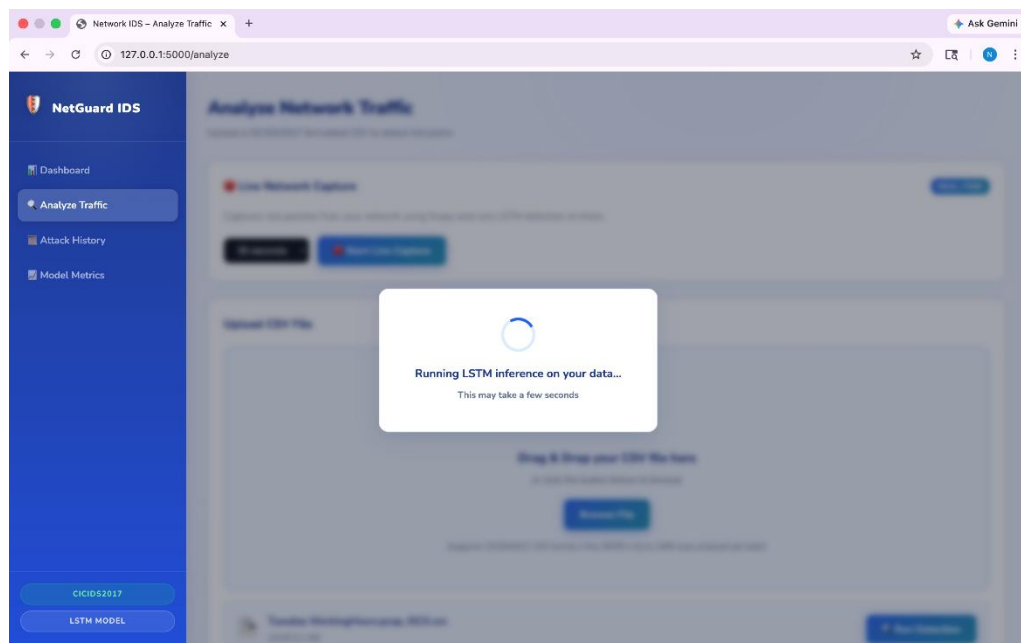
Screenshot 6.1



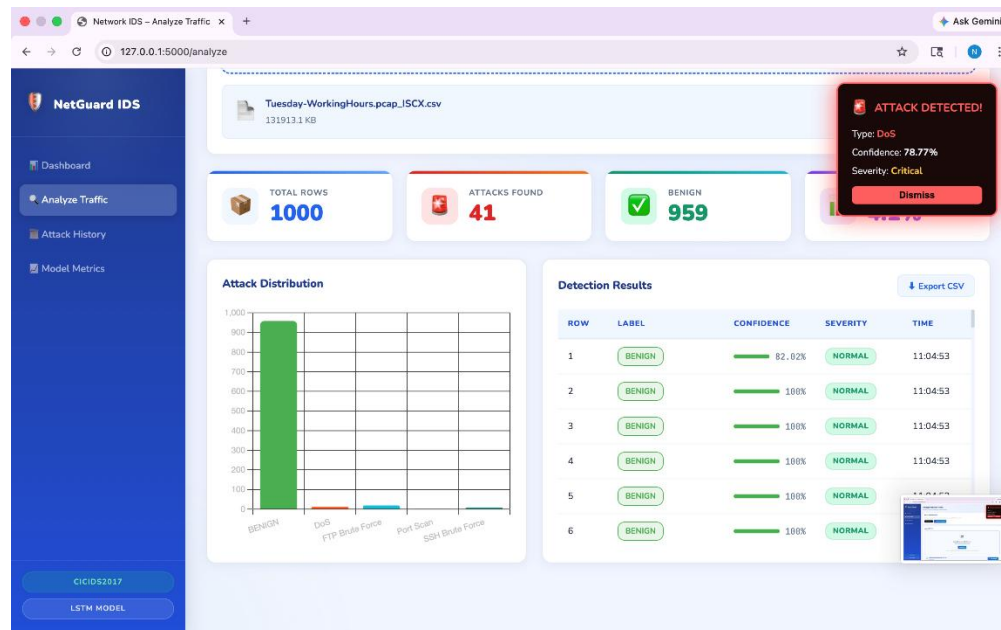
Screenshot 6.2



Screenshot 6.3



Screenshot 6.4



Screenshot 6.5

NetGuard IDS

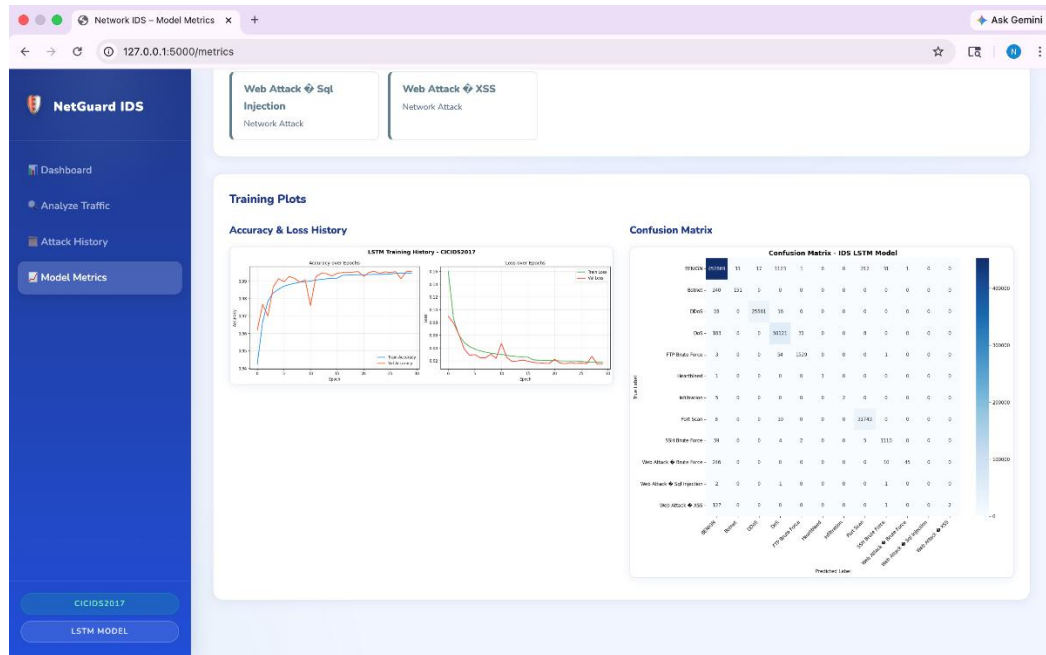
Attack History
All detections stored in MongoDB Atlas

Historical Attack Logs Showing last 100 records from MongoDB

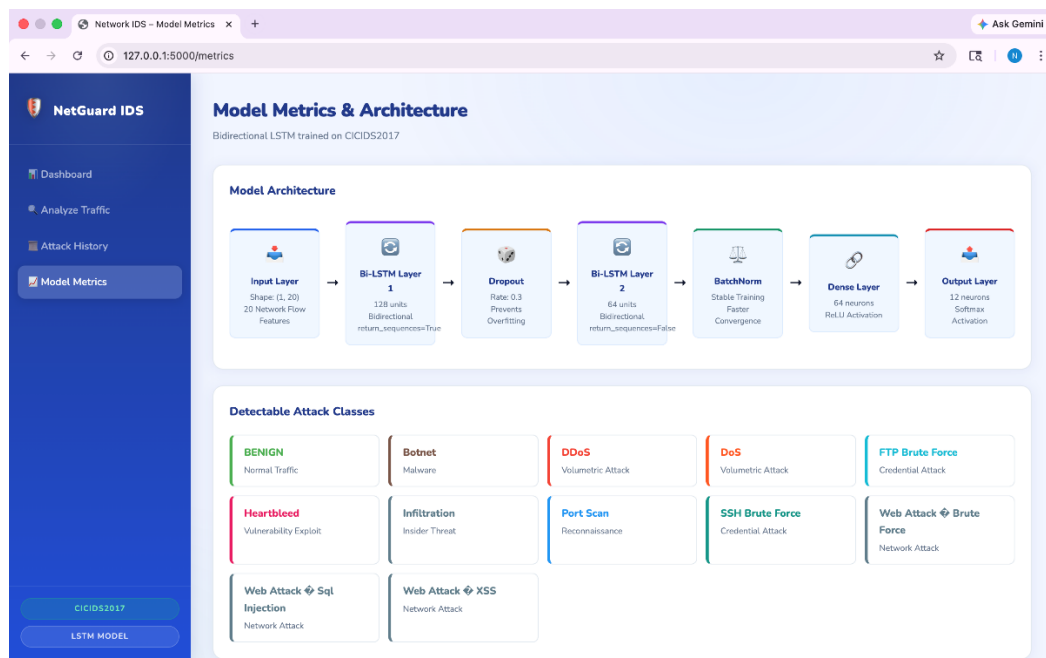
#	TIMESTAMP	IP ADDRESS	ATTACK TYPE	CONFIDENCE	SEVERITY	SOURCE
1	2026-03-30 11:06:14	127.0.0.1	BENIGN	100.0%	NORMAL	csv_upload
2	2026-03-30 11:06:14	127.0.0.1	BENIGN	100.0%	NORMAL	csv_upload
3	2026-03-30 11:06:13	127.0.0.1	BENIGN	100.0%	NORMAL	csv_upload
4	2026-03-30 11:06:13	127.0.0.1	BENIGN	100.0%	NORMAL	csv_upload
5	2026-03-30 11:06:13	127.0.0.1	BENIGN	100.0%	NORMAL	csv_upload
6	2026-03-30 11:06:13	127.0.0.1	BENIGN	78.29%	NORMAL	csv_upload

CICIDS2017
LSTM MODEL

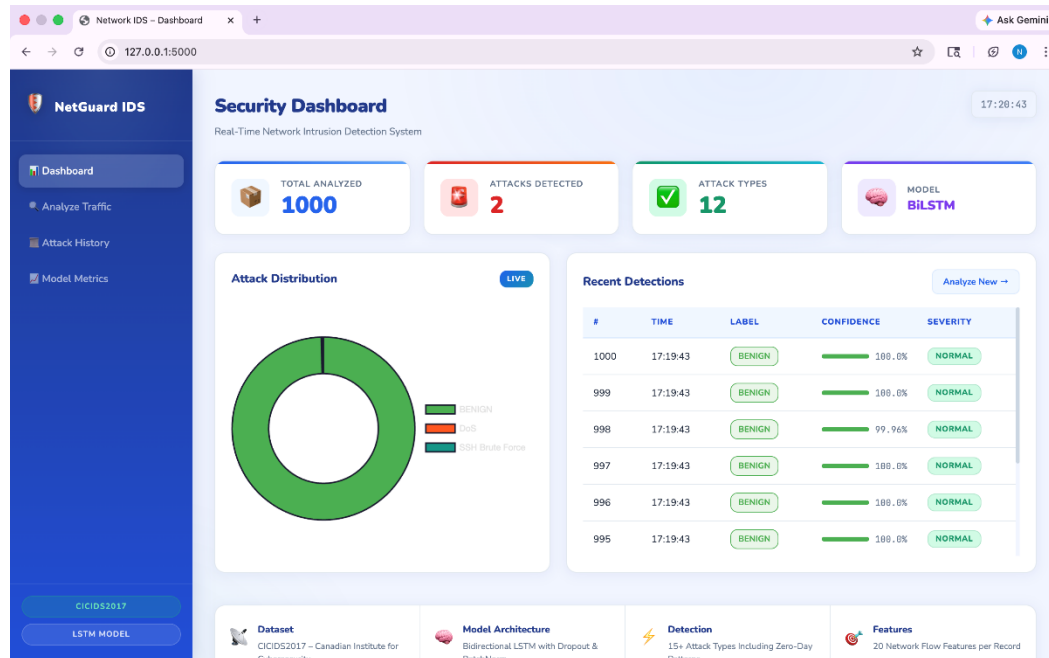
Screenshot 6.6



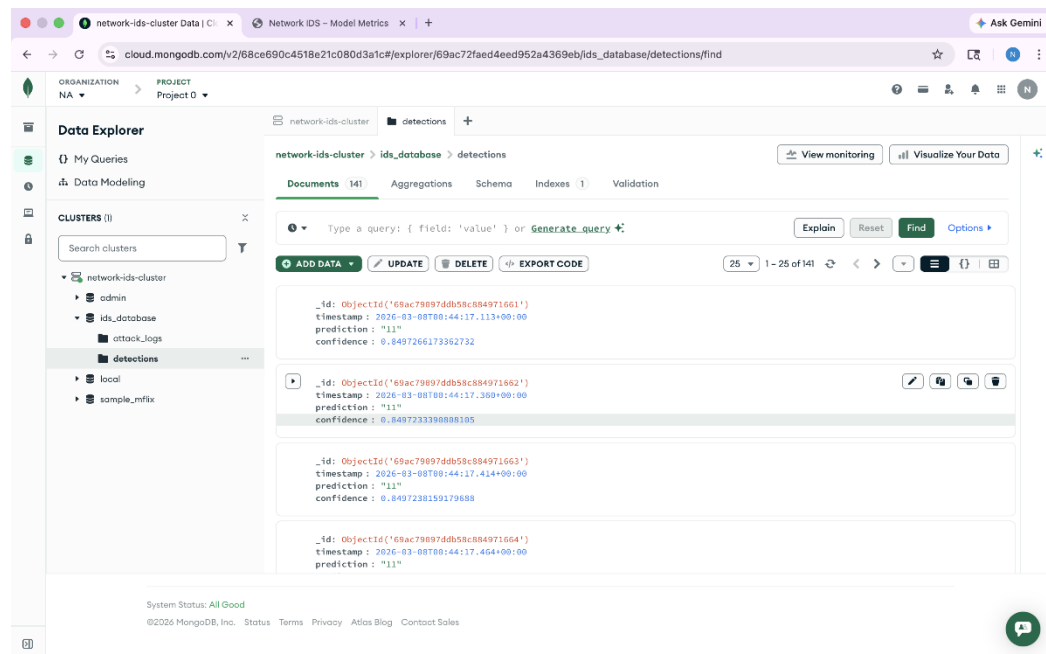
Screenshot 6.7



Screenshot 6.8



Screenshot 6.9



Screenshot 6.10

7. Test Cases

7.Test Cases

Test ID	Module	Test Scenario	Input	Expected Output	Actual output	Status
TC-01	Data Input	Upload valid CSV file	Correct CICIDS2017 CSV	File uploaded successfully	File uploaded successfully	Pass
TC-02	Data Input	Upload invalid file	Wrong format file	Error message displayed	Error message displayed	Pass
TC-03	Data Input	Start live capture	Capture request	Packets captured successfully	Packets captured successfully	Pass
TC-04	Preprocessing	Process dataset	Raw dataset	Cleaned and processed data	Data processed correctly	Pass
TC-05	Model	Run prediction	Valid input data	Correct attack classification	Correct classification	Pass
TC-06	Model	Detect attack	Malicious traffic	Attack detected with confidence	Attack detected	Pass
TC-07	Model	Detect normal traffic	Normal traffic	Classified as benign	Classified as benign	Pass
TC-08	Alert System	Generate alert	Attack detected	Alert displayed with severity	Alert displayed	Pass
TC-09	Database	Store results	Detection output	Data stored in MongoDB	Data stored	Pass
TC-10	Dashboard	View results	Detection data	Graphs and stats displayed	Displayed correctly	Pass
TC-11	Dashboard	View attack history	Stored records	History displayed	History displayed	Pass
TC-12	System	Real-time detection	Live traffic	Continuous detection	Working correctly	Pass

8. Conclusion

8. Conclusion

The project titled “Real-Time Network Intrusion Detection System Using Deep Neural Networks and Cloud Analytics” presents an effective and intelligent solution for enhancing network security through the use of advanced deep learning techniques. The system is designed to monitor network traffic continuously and detect various types of cyber-attacks in real time, thereby reducing the risk of unauthorized access and potential data breaches.

The implementation of a Bidirectional Long Short-Term Memory (BiLSTM) model plays a crucial role in analyzing sequential network data and identifying complex attack patterns. Unlike traditional methods, the deep learning approach enables the system to handle large volumes of data and accurately classify multiple attack types such as DoS, Probe, R2L, and U2R. The use of well-structured datasets like CICIDS2017 further improves the model’s performance by providing diverse and realistic network traffic scenarios.

The project also emphasizes the importance of data preprocessing and feature engineering, which ensure that raw network data is transformed into a suitable format for model training and prediction. Additionally, the integration of real-time packet capture using tools like Scapy allows the system to operate dynamically, making it capable of detecting threats as they occur.

Another significant aspect of the system is its real-time prediction engine, which connects live data with the trained model to provide instant results. The system not only identifies attacks but also generates alerts with confidence scores and severity levels, enabling users to take immediate action. The inclusion of an anomaly detection mechanism further enhances the system by identifying suspicious activities that may not be clearly classified by the model.

The use of cloud technologies such as MongoDB Atlas and cloud deployment platforms ensures scalability, reliability, and accessibility of the system. All detection results are securely stored in the database, allowing for historical analysis and forensic investigation. The dashboard and visualization module, developed using Flask and Plotly, provide an intuitive interface for users to monitor network activity, view attack trends, and analyze system performance.

Overall, this project successfully integrates concepts from cybersecurity, machine learning, and cloud computing to build a robust and scalable intrusion detection system. It demonstrates how modern technologies can be combined to create efficient and intelligent security solutions. The system not only improves detection accuracy but also provides real-time insights, making it highly relevant for practical applications in modern network environments.

9.Future Scope

9. Future Scope

The proposed system can be further improved by incorporating more advanced deep learning models such as transformers and hybrid models to enhance detection accuracy and efficiency. The system can be extended to support larger and more diverse datasets, enabling better generalization across different types of network environments.

Future enhancements may include the integration of automated response mechanisms, allowing the system to take immediate action against detected threats without manual intervention. The real-time prediction engine can also be optimized for faster processing and lower latency in high-traffic networks.

Additionally, deploying the system on scalable cloud infrastructures and integrating it with enterprise-level security tools such as firewalls and SIEM systems can improve its applicability in real-world scenarios. The inclusion of user authentication, role-based access control, and advanced visualization features can further enhance system usability and security.

10. References

10. References

1. NSL-KDD Dataset, “NSL-KDD Data Set for Network Intrusion Detection” available: <https://www.unb.ca/cic/datasets/nsf.html>
2. CICIDS2017 Dataset, Canadian Institute for Cybersecurity, Available: <https://www.unb.ca/cic/datasets/ids-2017.html>
3. TensorFlow Documentation, Available: <https://www.tensorflow.org>
4. Keras Documentation, Available: <https://keras.io>
5. Scapy Documentation, Available: <https://scapy.net>
6. Flask Documentation, Available: <https://flask.palletsprojects.com>
7. MongoDB Atlas Documentation, Available: <https://www.mongodb.com>
8. Plotly Documentation, Available: <https://plotly.com>
9. IEEE, Research papers on Intrusion Detection Systems, Available: <https://ieeexplore.ieee.org>
10. Google Scholar, Research articles on Deep Learning-based IDS, Available: <https://scholar.google.com>